

Presentation Script — Mini-Project 3

Total: ~15 minutes • Team: Jesse, Jake, Jeffrey

Speaker order: Jesse opens (slides 1-3) → Jake covers modeling (slides 4-7) → Jeffrey covers optimization (slides 8-9) → Jesse closes (slide 10) → Q&A.

SLIDE 1 — Title • Jesse • ~30 sec

Hi everyone, we're Jesse, Jake, and Jeffrey, and our project is *Predictive Modeling and Optimization for Real Estate Investment*.

The short version: we built a pipeline that combines a neural network and a linear program to help a real estate company decide which houses are worth investing in under a fixed renovation budget. I'll walk through the framing and the workflow, then Jake will cover the modeling side, and Jeffrey will cover the optimization. We'll close with limitations and take questions.

SLIDE 2 — The business problem • Jesse • ~1 min

The company we're consulting for has two connected questions, and neither one is interesting on its own.

The first is: *how much is each house worth?* That's a regression problem. The target is a continuous dollar amount, not a category, so we built a feed-forward neural network on the Ames Housing dataset to predict sale price from a small set of features.

The second question is: *which houses do we actually invest in?* The company has finite renovation capacity. They can't pursue every property. So we framed that side as a 0/1 knapsack — for each property, we either select it or skip it, and we want to maximize total expected gain without exceeding the budget.

The reason we need both is that prediction alone doesn't tell you what to buy — knowing a house is worth \$300K doesn't tell you whether to invest. And optimization alone doesn't know what each house is worth. The two pieces have to work together.

SLIDE 3 — The pipeline, end to end • Jesse • ~45 sec

Here's the workflow in three stages.

We start with the Ames Housing dataset — 2,930 properties, 82 features. From there, we train a

neural network to predict sale price. Then we feed those predictions into a linear program that picks the optimal subset of properties under a renovation budget.

The output of stage two is a predicted price and an expected gain for every house. The output of stage three is the actual investment portfolio — a yes-or-no decision for each property.

I'll hand it over to Jake to walk through stages one and two.

SLIDE 4 — Eight features, one swap · *Jake* · ~1 min 30 sec

Thanks. The project allows up to eight input variables, so we picked eight that cover the main drivers of house price.

Overall Quality is the strongest single predictor — material and finish quality, correlation 0.80 with sale price. Then we have Above-Grade Living Area and Total Basement Square Footage for size. Garage Cars captures garage capacity. Year Built captures age. Full Bath is a buyer-facing feature. And Neighborhood — location — is the only categorical, one-hot encoded into 28 binary columns.

The eighth feature is Year Remod/Add, which is when the house was last renovated. We originally picked Lot Area instead, but when we looked at the distribution it was heavily right-skewed and only weakly correlated with price — about 0.27. Year Remod/Add was much better behaved, with correlation 0.53, and it ties directly into the renovation framing of Part 4. So the swap was driven by methodology, not raw accuracy. Both versions performed within noise on validation.

SLIDE 5 — Splits, scaling, and a baseline to beat · *Jake* · ~1 min

Quick on data prep. We dropped two rows with missing values out of 2,930 — negligible loss. One-hot encoded Neighborhood, standard-scaled the numeric features, and standard-scaled the target internally so the network trains on a well-behaved range.

Splits are 64% train, 16% val, 20% test — that's two nested 80/20 splits with a fixed seed. We use the validation set to compare architectures and tune hyperparameters, so we never peek at test until the final report.

For the metric, we report RMSE in dollars because it's interpretable and penalizes big misses, which matter for investment decisions. MAE and R-squared are reported alongside.

The baseline is the simplest possible model: predict the training mean for every test row. That gives us an RMSE of about \$90K and an R-squared near zero. Any real model has to crush this — and ours did.

SLIDE 6 — Four architectures, one winner • *Jake* • ~1 min 30 sec

We trained four neural network architectures using scikit-learn's MLPRegressor, all with ReLU, Adam optimizer, and early stopping.

NN-A is a single 32-neuron layer. NN-B has two layers — 64 and 32. NN-C is a wide single layer with 128 neurons. NN-D has three layers — 64, 32, 16.

All four crush the baseline and cluster between R-squared 0.86 and 0.89 on test. Two things stand out. First, depth slightly beats width — the wide single-layer NN-C was actually the worst on validation. Second, going past two layers doesn't help — NN-D added a third layer for no gain. And we don't see overfitting in any of them, because val and test scores match closely throughout.

We picked NN-B as the winner based on validation performance — it had the best val R-squared. On the test set, that translated to R-squared 0.887 and an RMSE of about \$30K.

SLIDE 7 — Tuning NN-B: gains were marginal • *Jake* • ~1 min

We then ran a 12-combo hyperparameter grid on NN-B, sweeping the L2 regularization and Adam learning rate.

The best combination — alpha 0.1 and learning rate 0.01 — bumped val R-squared from 0.876 to 0.883. But on test, it went from 0.887 to 0.884. Essentially flat, within noise.

The takeaway: the model is data-bounded, not tuning-bounded. The grid was mostly flat across the board, which means more careful hyperparameter sweeps won't push us past R-squared 0.89. To do that, we'd need more features or a different model class — gradient boosting would be a natural next step. I'll hand it over to Jeffrey for the optimization.

SLIDE 8 — From predictions to decisions • *Jeffrey* • ~2 min

Thanks Jake. So we have a trained neural network that predicts sale price. Now we use those predictions to make actual investment decisions.

The optimization runs on the 586 test-set properties — the same houses the neural network never saw during training. That's important: if we used training rows, the predictions would be artificially perfect and we'd be conflating model fit with real out-of-sample signal.

For each property we compute two quantities. The first is *Expected Gain* — that's the predicted price minus the actual sale price. So it's the NN's residual. A positive value means the model thinks the house is priced below its typical pattern — which flags it as a candidate worth

investigating. A negative value means the opposite.

The second quantity is *Renovation Burden* — a proxy for how much work each property would take. It combines three real Ames features: condition, time since the last remodel, and size. We z-score each component before adding them so they contribute on equal footing — without z-scoring, size in square feet would dominate condition on a 1-to-10 scale.

The budget is data-driven. We assume the company can handle 10% of the total renovation workload across all 586 candidates, so the budget scales with the actual mix of properties.

The LP itself is a 0/1 knapsack: for each property we decide to invest or not, the objective maximizes total Expected Gain, the constraint keeps total burden under budget, and we use PuLP with the CBC solver — checking for Optimal status before trusting any results.

SLIDE 9 — 60 candidates, \$2.38M expected gain • Jeffrey • ~2 min

Here are the results.

The LP selected 60 of 586 candidates — about 10% of the pool — for a total Expected Gain of \$2.38 million, roughly \$40K per pick on average. The capacity constraint binds at 100%, meaning every available unit of renovation budget is used.

Looking at what the portfolio actually looks like: the picks lean toward newer remodels — 31 years on average versus 42 dataset-wide. That's because the age term in the burden formula systematically pushes toward more-recently-modernized homes.

Condition isn't the differentiator — both selected and dataset-wide average is 5.5 out of 10. So the age and size terms in the burden formula do the filtering, not condition.

The selected portfolio is slightly larger on average — 1,700 square feet versus 1,513 — but that's actually skewed by one outlier at 5,095 square feet. That property has the maximum burden in the entire dataset, but the LP took it anyway because its expected gain is \$215K — about 5x the next-best gain.

That tells us something important about how the LP is making decisions: it's not picking the highest-gain properties, and it's not picking the lowest-burden properties. It's optimizing gain-per-unit-burden. So a high-burden outlier can win when its gain is large enough to justify the cost.

One honest caveat we want to flag: Expected Gain is the NN's residual on out-of-sample data. The NN sees the same features the market sees — size, quality, location — so it can't reveal information the market doesn't already price in. Treat the LP output as a candidate list to investigate further, not a final buy list.

I'll hand it back to Jesse to close.

SLIDE 10 — What we'd tighten with more time · Jesse · ~1 min 30 sec

Thanks Jeffrey. Four limitations worth flagging.

First — single random seed. Every comparison we ran used the same seed. The top models differ by less than 0.01 R-squared, which is in noise range. Running three to five seeds would tell us whether NN-B truly beats NN-A or just got lucky on this split.

Second — Expected Gain conflates prediction error with mispricing. A property with a large positive gain might be a real bargain, or the NN might just be wrong about it. With \$30K test RMSE, that uncertainty on individual gains is real.

Third — the burden formula has no dollar grounding. It captures the right intuitions, but it's not connected to actual construction cost. A real engagement would tighten this with construction-cost data.

Fourth — no confidence intervals. Both Expected Gain and Burden are point estimates. The model's RMSE propagates into the portfolio total but isn't quantified.

The bottom line: prediction tells us what each house is worth relative to its features. Optimization tells us which undervalued houses we can actually afford under capacity limits. The two pieces have to work together. Thanks for listening — we're happy to take questions.

Likely Q&A

Q: Why a neural network instead of something simpler like linear regression or gradient boosting? *Jake:* The project required a neural network. That said, gradient boosting would probably outperform on this dataset size — Ames is small enough that XGBoost or LightGBM tend to dominate. The NN got us to R^2 0.89 with eight features, which we think is reasonable for the assignment.

Q: Why didn't you tune more hyperparameters? *Jake:* Time constraint, mostly. We deliberately kept the grid small — alpha and learning rate — because the gains were already marginal. With more time, we'd try lbfgs as a solver instead of Adam, since lbfgs tends to outperform on small datasets like this one.

Q: Why z-score the burden components? *Jeffrey:* Without z-scoring, the size term in square feet would completely dominate condition on a 1-to-10 scale. Z-scoring puts each component on the same footing so the formula reflects the design intent — that all three matter equally — rather than letting raw scale do the work.

Q: Is the 10% budget arbitrary? *Jeffrey:* It's a calibration choice, but it's not arbitrary in the

sense that it's tied to the data. We're saying the company handles 10% of the total renovation workload of the candidate pool. If the pool changes, the budget rescales automatically. A real engagement would replace this with the company's actual capacity number.

Q: What happens if you change the budget? *Jeffrey:* The profit-vs-budget curve is roughly linear in this range — we have a chart of it. As the budget grows, the LP picks more properties and total gain grows. There isn't a clear concave point where additional budget stops paying off, at least within the range we tested.

Q: How do you handle properties with negative Expected Gain? *Jeffrey:* The LP avoids them automatically when maximizing — they'd hurt the objective. There's no explicit filter, but in practice nothing with a negative gain gets selected.

Q: Why use the test set for the LP instead of all 2,928 rows? *Jeffrey:* If we used training rows, the NN's predictions would be near-perfect on those rows because it had already seen them — that would inflate Expected Gain artificially. Using only test-set rows keeps the residuals truly out-of-sample, which is the only honest way to do this.